

Методы оптимизации. Семинар 3. Autodiff.

Корнилов Никита Максимович

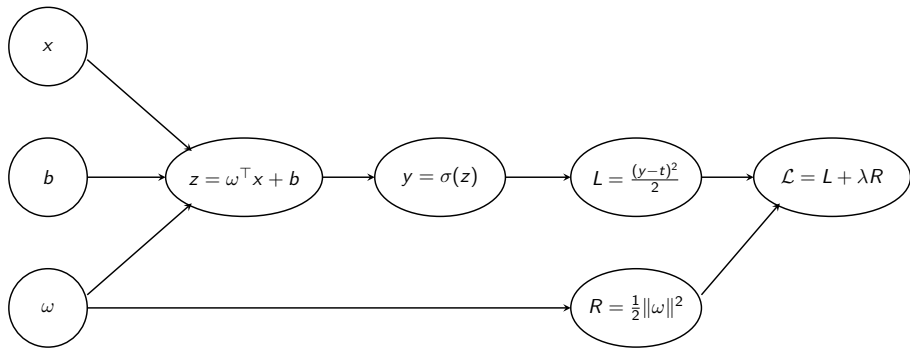
МФТИ ФИВТ

24 сентября 2026г

Функции представляют собой последовательность (дифференцируемых) параметрических преобразований. Строим вычислительный граф (computational graph), где промежуточным вершинам соответствуют преобразования, входящим стрелкам – входные переменные, а выходным стрелкам – результат преобразования.

$$f(x, \omega, b) = \frac{(\sigma(\omega^\top x + b) - t)^2}{2} + \lambda \cdot \frac{1}{2} \|\omega\|_2^2, \quad x, \omega \in \mathbb{R}^n, b \in \mathbb{R}.$$

$$f(x, \omega, b) = \frac{(\sigma(\omega^\top x + b) - t)^2}{2} + \lambda \cdot \frac{1}{2} \|\omega\|_2^2, \quad x, \omega \in \mathbb{R}^n, b \in \mathbb{R}.$$



Подсчёт значения $f(x)$ по графу называется forward pass.

Мы хотим найти градиент по переменной $x \in \mathbb{R}^n$ финальной функции $f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$.

Back Propagation

Идти от детей к родителям и в вершине u считать $\frac{\partial f}{\partial u}$.

Для вершины u с детьми $v_1, \dots, v_d$ мы имеем $f(u(x)) = f(v_1(u(x)), \dots, v_d(u(x)))$ и по правилу дифф функции нескольких переменных:

$$\frac{\partial f}{\partial u}(x) = \sum_{j=1}^d \frac{\partial f}{\partial v_j}(x) \cdot \frac{\partial v_j}{\partial u}(x), \quad \frac{\partial f}{\partial v_j}(x) = (\nabla_{v_j} f(x))^T.$$

В конце мы доходим до начальной вершины x и получаем $\frac{\partial f}{\partial x}$.

Back Propagation

Отсортируем все m вершин от родителей к детям и перенумеруем их от 1 до m .

Обозначим производную функции f по вершине u_i как

$$\overline{u_i} = \frac{\partial f}{\partial u_i}.$$

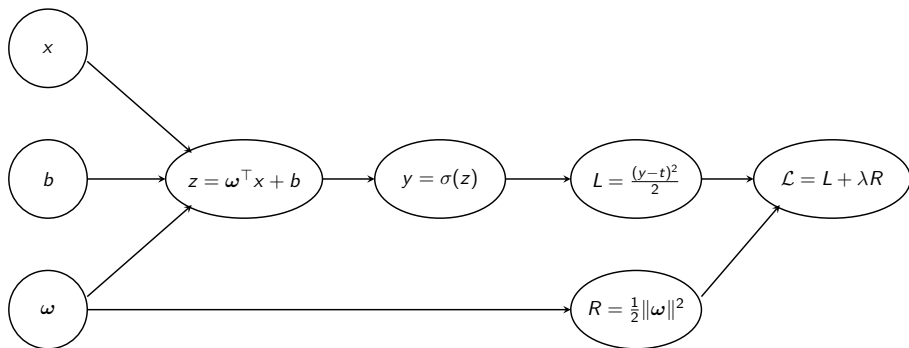
Общий алгоритм действий выглядит так

- 1 Произвести forward pass и сохранить все значения u_i как функции от их родителей.
- 2 Положить $\overline{u_m} = 1$ и для всех $i = m - 1, \dots, 1$ посчитать

$$\overline{u_i} = \sum_{j \in \text{дети}(u_i)} \overline{u_j} \frac{\partial u_j}{\partial u_i}.$$

$$f(u) = f(v_1(u), \dots, v_d(u))$$

$$\frac{\partial f}{\partial u} = \sum_{j=1}^d \frac{\partial f}{\partial v_j} \cdot \frac{\partial v_j}{\partial u}.$$



$$f(u) = f(v_1(u), \dots, v_d(u))$$

$$\frac{\partial f}{\partial u} = \sum_{j=1}^d \frac{\partial f}{\partial v_j} \cdot \frac{\partial v_j}{\partial u}.$$

- 1 Необходимо хранить ВСЕ промежуточные значения u_i .

$$f(u) = f(v_1(u), \dots, v_d(u))$$

$$\frac{\partial f}{\partial u} = \sum_{j=1}^d \frac{\partial f}{\partial v_j} \cdot \frac{\partial v_j}{\partial u}.$$

- 1 Необходимо хранить ВСЕ промежуточные значения u_i .
- 2 Важно уметь быстро превращать градиент по выходу в градиент по входу (умножать матрицу Якоби $\frac{\partial v_j}{\partial u}$ на строку $\frac{\partial f}{\partial v_j}$ слева).
- 3 Операции с матрицами Якоби эффективно разработаны в рамках библиотек автоматического дифференцирования.

$$f(u) = f(v_1(u), \dots, v_d(u))$$

$$\frac{\partial f}{\partial u} = \sum_{j=1}^d \frac{\partial f}{\partial v_j} \cdot \frac{\partial v_j}{\partial u}.$$

- 1 Необходимо хранить ВСЕ промежуточные значения u_i .
- 2 Важно уметь быстро превращать градиент по выходу в градиент по входу (умножать матрицу Якоби $\frac{\partial v_j}{\partial u}$ на строку $\frac{\partial f}{\partial v_j}$ слева).
- 3 Операции с матрицами Якоби эффективно разработаны в рамках библиотек автоматического дифференцирования.
- 4 Каждая вершина может быть запрограммирована как отдельная сущность, умеющая внутри себя делать forward и backward pass.

$$f(u) = f(v_1(u), \dots, v_d(u))$$

$$\frac{\partial f}{\partial u} = \sum_{j=1}^d \frac{\partial f}{\partial v_j} \cdot \frac{\partial v_j}{\partial u}.$$

- 1 Необходимо хранить ВСЕ промежуточные значения u_i .
- 2 Важно уметь быстро превращать градиент по выходу в градиент по входу (умножать матрицу Якоби $\frac{\partial v_j}{\partial u}$ на строку $\frac{\partial f}{\partial v_j}$ слева).
- 3 Операции с матрицами Якоби эффективно разработаны в рамках библиотек автоматического дифференцирования.
- 4 Каждая вершина может быть запрограммирована как отдельная сущность, умеющая внутри себя делать forward и backward pass.
- 5 Можно строить граф вычисления производной.

Умножение матрицы Якоби на строку

Для функции вида $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ и вектора $y \in \mathbb{R}^m$ считаем значение $J_f^\top(x) \cdot y$ в точке x , а именно

$$d\langle y, f(x) \rangle = \langle y, J_f dx \rangle = \langle J_f^\top y, dx \rangle = \langle \nabla g(x), dx \rangle \quad \Rightarrow \quad J_f^\top y = \nabla g(x),$$

где $g(x) = \langle f(x), y \rangle$.

Вместо полного гессиана можно найти градиент функции g . Именно поэтому вычисления из Back Propagation можно эффективно реализовать на практике.

Умножение гессиана на вектор

Дважды непрерывно дифференцируемая функция $f : \mathbb{R}^n \rightarrow \mathbb{R}$, вектор $y \in \mathbb{R}^n$.

Покажем, как можно эффективно считать $\nabla^2 f(x) \cdot y$:

$$d(\langle \nabla f(x), y \rangle) = d(\nabla f)^\top y = dx^\top J_{\nabla f}^\top \cdot y = dx^\top \nabla^2 f(x) \cdot y = dx^\top \nabla g(x),$$

где $g(x) = \langle \nabla f(x), y \rangle$.

Вместо полного гессиана можно найти градиент функции g .

Forward Propagation

Идти от родителей к детям и в вершине u считать $\frac{\partial u}{\partial x}$.

Для вершины u с родителями $p_1, \dots, p_k$ мы имеем $u(x) = u(p_1(x), \dots, p_k(x))$ и по правилу дифф функции нескольких переменных:

$$\frac{\partial u}{\partial x}(x) = \sum_{j=1}^k \frac{\partial u}{\partial p_j}(x) \cdot \frac{\partial p_j}{\partial x}(x).$$

В конце мы доходим до конечной вершины f и получаем $\frac{\partial f}{\partial x}$.

Обсуждение Forward Propagation

$$u = u(p_1, \dots, p_k)$$
$$\frac{\partial u}{\partial x} = \sum_{j=1}^k \frac{\partial u}{\partial p_j} \cdot \frac{\partial p_j}{\partial x}.$$

- Можно совершать forward pass вместе с подсчётом $\frac{\partial u}{\partial x}$ и нужно будет хранить только один слой графа.
- В forward propagation нужно хранить $\frac{\partial u_i}{\partial x}$, а в backward propagation $\frac{\partial f}{\partial u_i}$. Если размерность входа x намного больше, чем размерность выхода $f(x)$, то на каждом шаге forward pass нужно будет хранить настолько же больше данных.

$$u = u(p_1, \dots, p_k)$$
$$\frac{\partial u}{\partial x} = \sum_{j=1}^k \frac{\partial u}{\partial p_j} \cdot \frac{\partial p_j}{\partial x}.$$

- Можно совершать forward pass вместе с подсчётом $\frac{\partial u}{\partial x}$ и нужно будет хранить только один слой графа.
- В forward propagation нужно хранить $\frac{\partial u_i}{\partial x}$, а в backward propagation $\frac{\partial f}{\partial u_i}$. Если размерность входа x намного больше, чем размерность выхода $f(x)$, то на каждом шаге forward pass нужно будет хранить настолько же больше данных.

Вывод

Для $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ выгоднее использовать

- Back Propagation при $n \geq m$,
- Forward Propagation при $n \ll m$.